

A memetic algorithm for job shop scheduling using a critical-path-based local search heuristic

Mohammad R. Raeesi N. · Ziad Kobti

Received: 26 July 2011 / Accepted: 23 May 2012 / Published online: 12 June 2012
© Springer-Verlag 2012

Abstract In this article, a new memetic algorithm has been proposed to solve job shop scheduling problems (JSSPs). The proposed method is a genetic-algorithm-based approach combined with a local search heuristic. The proposed local search heuristic is based on critical operations. It removes the critical operations and reassigns them to a new position to improve the fitness value of the schedule. Moreover, in this article, a new fitness function is introduced for JSSPs. The new fitness function called priority-based fitness function is defined in three priority levels to improve the selection procedure. To show the generality of our proposed method, we apply it to three different types of job scheduling problems including classical, flexible and multi-objective flexible JSSPs. The experiment results show the efficiency of the proposed fitness function. In addition, the results show that incorporating local search not only offers better solutions but also improves the convergence rate. Compared to the state-of-the-art algorithms, the proposed method outperforms the existing methods in classical JSSPs and offers competitive solutions in other types of scheduling problems.

Keywords Memetic algorithm · Job shop scheduling · Priority-based fitness function · Critical operations

1 Introduction

Job shop scheduling is the process of scheduling various jobs between different machines in a manufacturing system. In the

last decades, a lot of work has been done to control a job shop system. There are different types of job shop scheduling such as classical and flexible. Each job shop includes different jobs such that each job has a fixed sequence of operations to be processed in order. The number of operations for different jobs could also be different. Each operation has to be processed only on one machine, while different operations of a job could be processed on different machines.

In the classical job shop scheduling problem (JSSP), each operation can be processed only on a specific machine. It means that there is no machine selection flexibility for operations in JSSP. So, JSSP is the process of determining the sequence of operations of different jobs which have to be processed on the same machine. In other words, JSSP determines an ordered list of operations for each machine. Unlike JSSP, in flexible job shop scheduling (FJSS), there is the flexibility for each operation to be processed on different machines. One of the first research work which considered flexibility in job shop scheduling was published by Bruker and Schlie [6]. If all the operations have the flexibility of being processed on all available machines, then the problem is called total flexible job shop scheduling (T-FJSS). Otherwise, it is called partial flexible job shop scheduling (P-FJSS). Considering the computational cost and search space, Zhang et al. [45] mentioned that the P-FJSS is more complex than T-FJSS. Few researchers also considered the uncertainty for FJSS such as stochastic processing time, and called problem as stochastic flexible job shop (SFJS). Mahdavi et al. [29] proposed a simulation-based decision support system (DSS) to control the stochastic job shop system.

However, it is clear that FJSS is more complex than JSSP; in JSSP there is only the operation sequencing problem for each machine, while in FJSS, it is combined with the machine selection problem as well. Some researchers considered both machine selection and operation sequencing as one problem,

M. R. Raeesi N. (✉) · Z. Kobti
School of Computer Science, University of Windsor,
Windsor, ON N9B-3P4, Canada
e-mail: raeesim@uwindsor.ca

Z. Kobti
e-mail: kobti@uwindsor.ca

such as Hurink et al. [22], while others solved them as separate problems such as Brandimarte [5].

Open shop scheduling is something different than job shop scheduling. The main difference is that in open shop scheduling operations of a job are sequence independent, while in job shop scheduling the sequence of operations for each job is fixed. Naderi et al. [34] proposed a memetic algorithm for open shop scheduling.

The main goal of JSSPs is to utilize the manufacturing system. Each manufacturing system has its own objective. In most cases, the objective of the scheduling is to minimize the maximum completion time, also called *makespan*. Moreover, there are some systems looking for various parameters as their objectives, which are called multi-objective flexible job shop scheduling (MO-FJSS). For instance, the objective of a system can be both minimizing maximum machine workload and maximizing machine utilization. To deal with such systems, some researchers such as Mahdavi et al. [29] used generalized response surface methodology, in which one response is the goal and others are considered as constraint variables.

In this article, a memetic algorithm is proposed to solve scheduling problems. The proposed algorithm is applied to different types of JSSPs including JSSP, FJSS and MO-FJSS problems. It should be mentioned that stochastic parameters are beyond the scope of this work and not considered to evaluate our method in this article, but these can be considered in future work. The outline of this paper starts with the related work in Sect. 2, followed by a more detailed treatment of the problem definition in Sect. 3. Section 4 describes the proposed memetic algorithm in details. The evaluation results of our proposed algorithm are discussed in Sect. 5. Finally, conclusions are presented in Sect. 6.

2 Related works

Job shop scheduling is well known in different fields, and gained popularity mainly because of its application in various manufacturing systems. It is proved by Garey et al. [14] that job shop scheduling for systems with more than two machines are NP-complete problems which means there is no exact algorithm able to find the best schedule for all types of problems.

There are various types of algorithms proposed to solve JSSPs, including heuristics, and meta-heuristics. Bottleneck shifting procedure used by Huang and Yin [21] and tree search method presented by Asano and Ohta [2] are instances of heuristic approaches. Meta-heuristic approaches include Simulated Annealing (SA), Tabu Search (TS), Ant Colony Optimization (ACO), Genetic Algorithm (GA), hybrid GA, and so on. Kolonko [25] was one of the first researchers to apply Simulated Annealing (SA) on job shop scheduling. A

combination of Tabu Search and Bottleneck shifting is used by Pezzella and Merelli [39]. Wang et al. [41] proposed an ACO approach for JSSP. A combination of ACO and TS is also proposed by Huang and Liao [20].

One of the most widely used algorithms in job shop scheduling is GA either as a single procedure or combined with a local search (LS) heuristic. The application of GA to job shop scheduling was first introduced by Lawrence [26]. Four more different GAs were also presented by Chen et al. [8], Mattfeld and Bierwirth [30], Pezzella et al. [40] and Zhang et al. [45] as a single efficient procedure to solve the JSSP.

Most of the recent work in job shop scheduling hybridized GAs with LS algorithms to find better results as well as to improve its convergence rate. Moscato [32] defined memetic algorithm (MA) as a population-based global search method combined with a local search heuristic to be used to solve an optimization problem. Ong and Keane [36] considered population-based search as exploration to find the best region in the search space, and local search as exploitation to find the best individual in a neighborhood. Based on this definition, all the hybridizations of GAs or other Evolutionary Algorithms (EAs) with LS heuristics can be considered as MAs. Park et al. [38], Chiang et al. [9], Gao et al. [12, 13], and Caumond et al. [7] proposed different MAs with different heuristics.

The MA proposed by Yang et al. [44] is a combination of Clonal Selection and SA. A combination of GA and Tabu Search was also proposed by Ombuki and Ventresca [35]. Kacem et al. [23] combined an EA with a localization approach. They also proposed a combination of EA with fuzzy logic [24]. Hassan et al. [16–18] proposed different GAs combined with different priority rules. Recently, Hasan et al. [19] proposed three different MAs which are combinations of GAs with three different priority rules. Among these three, they claimed that the most effective priority rule is joint of gap reduction and restricted swapping. Gap reduction is the rule of moving operations to the first appropriate existing gaps in which the corresponding machine is idle. Restricted swapping rule tries to swap the adjacent operations on a machine to decrease the makespan.

Moreover, there are more different algorithms applied in job shop scheduling such as Neural Network algorithm used by Fonseca and Navaresse [11], and artificial immune algorithms (AIA) proposed by Bagheri et al. [3] and Mobini et al. [31]. A simulation modeling algorithm is proposed by Xing et al. [43]. Mahdavi et al. [29] combined the simulation modeling with a DSS to improve the final results.

Recently, we proposed a MA to deal with classical JSSPs, in which we introduced a new chromosome representation called Machine Operation List (MOL) [33]. Since MOL is based on the operations on their applicable machine, it is appropriate only for classical JSSP. Therefore, it has not been used in this article. The local search of our previous work

deals with critical jobs and their operations, while here a new concept is introduced which is selected critical operations.

3 Problem definitions

In all types of JSSPs, there are a set of machines and a set of jobs to be processed on those machines. Machines are denoted by m_k , where k refers to the index of each machine which is between 1 to M , the number of machines. Jobs are denoted by J_i , where i refers to the job index ranging from 1 to N , the total jobs number. Each job consists of a number of operations, denoted by NO_i . The operations are denoted by O_{ij} , meaning the j th operation of the i th job. The processing time of each operation is known and denoted by $PT(O_{ij}, m_k)$, meaning the processing time of operation O_{ij} on k th machine. It should be mentioned that the moving time for jobs between machines and the set up time for each machine are assumed to be negligible. Table 1 shows the 4th benchmark of Xing et al. [43] which is a sample data for the T-FJSS. In this case, there are 10 jobs with the same number of operations as 3 operations per job to be processed on 10 different machines. The reason for representing this sample is that our proposed method outperforms Xing et al. [43] approach on this test problem. The improved result are shown in Fig. 8.

The rules of the job shop scheduling are as follows: The sequence of operation for each job is fixed. For example, the third operation of a job cannot be processed until the second operation has been done. However, the operations of different jobs are independent. At any given time, only one operation of a job can be processed on one and only one machine. It is considered that all the jobs and all the machines are available at the starting time.

It should be mentioned here that there are two types of operation sequences: job-operation sequence and machine-operation sequence. Job-operation sequence is the fixed sequence of operations for each job. So the job-preceding operation of an operation is its preceding operation in job-operation sequence. Machine-operation sequence is the sequence of operations which are processed on the same machine. The machine-preceding operation of an operation is its preceding operation which is processed on the same machine exactly before it.

4 Proposed memetic algorithm

The proposed MA is a combination of a GA and a LS heuristic, which is the main contribution of this article. The proposed GA is a simple GA with a sophisticated fitness function. In each generation after fitness evaluation the LS is applied to some selected individuals. The heuristic used in LS is moving critical operations to improve the fitness

Table 1 4th FJSS benchmark used by Xing et al. [43]

Jobs	Operations	Machines									
		m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}
J_1	O_{11}	1	4	6	9	3	5	2	8	9	5
	O_{12}	4	1	1	3	4	8	10	4	11	4
	O_{13}	3	2	5	1	5	6	9	5	10	3
J_2	O_{21}	2	10	4	5	9	8	4	15	8	4
	O_{22}	4	8	7	1	9	6	1	10	7	1
	O_{23}	6	11	2	7	5	3	5	14	9	2
J_3	O_{31}	8	5	8	9	4	3	5	3	8	1
	O_{32}	9	3	6	1	2	6	4	1	7	2
	O_{33}	7	1	8	5	4	9	1	2	3	4
J_4	O_{41}	5	10	6	4	9	5	1	7	1	6
	O_{42}	4	2	3	8	7	4	6	9	8	4
	O_{43}	7	3	12	1	6	5	8	3	5	2
J_5	O_{51}	7	10	4	5	6	3	5	15	2	6
	O_{52}	5	6	3	9	8	2	8	6	1	7
	O_{53}	6	1	4	1	10	4	3	11	13	9
J_6	O_{61}	8	9	10	8	4	2	7	8	3	10
	O_{62}	7	3	12	5	4	3	6	9	2	15
	O_{63}	4	7	3	6	3	4	1	5	1	11
J_7	O_{71}	1	7	8	3	4	9	4	13	10	7
	O_{72}	3	8	1	2	3	6	11	2	13	3
	O_{73}	5	4	2	1	2	1	8	14	5	7
J_8	O_{81}	5	7	11	3	2	9	8	5	12	8
	O_{82}	8	3	10	7	5	13	4	6	8	4
	O_{83}	6	2	13	5	4	3	5	7	9	5
J_9	O_{91}	3	9	1	3	8	1	6	7	5	4
	O_{92}	4	6	2	5	7	3	1	9	6	7
	O_{93}	8	5	4	8	6	1	2	3	10	12
J_{10}	O_{101}	4	3	1	6	7	1	2	6	20	6
	O_{102}	3	1	8	1	9	4	1	4	17	15
	O_{103}	9	2	4	2	3	5	2	4	10	23

value. Moving critical operations introduced in the literature is restricted such that it only tries to find an idle time interval, while in our proposed method, rescheduling is used to find the best positions. Furthermore, the proposed method is a generic method applicable for various types of job shop scheduling such as JSSP, FJSS and MO-FJSS.

4.1 Chromosome representation

In the literature, there are various chromosome representations proposed by different researchers. In this article, MSOS representation proposed by Zhang et al. [45] has been used, because it is efficient for coding and decoding procedures. This representation always generates feasible schedules so repair mechanisms are not required. MSOS representation

includes two parts: Machine Selection (MS) and Operation Sequence (OS).

Machine Selection determines the selected machine for each operation among its applicable machines. Hence it is a string of machines indices with the length of total number of operations. For example, if an operation has the flexibility to be processed on machine 2 and machine 4, number 1 for its corresponding MS entry means the 2nd machine, and 2 means the 4th one. Therefore, in JSSP where there is no flexibility for machine selection, the MS is always a string of 1s.

Operation Sequence is also a string of numbers determining the sequence of operations. In this string the operations are denoted by their job indices and the occurrence of job indices determines the index of each operation for each job. Consider the Operation Sequence

$$OS = [3, 2, 3, 1, 2, 2, 1]$$

which represents the following sequence of operations.

$$O_{31} < O_{21} < O_{32} < O_{11} < O_{22} < O_{23} < O_{12}$$

Since the operations are denoted by their job indices and the operation indices are determined automatically, all the variations of OS is thus feasible and there is no need for any repair mechanism.

In chromosome decoding, we aim to generate only active schedules to limit the search space. Active schedules are ones where there is no operation to be processed earlier without delaying another operation. By assigning the operations to the first appropriate intervals, the resulting schedule would be active. By this definition, an optimal schedule is either an active schedule or has an equivalent active schedule, so it can be reached by exploring only the active schedules. Active schedule concept is introduced by Croce et al. [10] and used by Becerra and Coello [4] and Hasan et al. [19] as permissible left shift and gap reduction rule, respectively.

4.2 Genetic operators

The characteristics of each GA are the initial generation, the crossover operation and the mutation method. The initial generation for MS is exactly the same as what Zhang et al. [45] used. It includes all the global selection, local selection and random selection with the same percentage as their suggestion. The OS part of the initial chromosomes are produced by Long Job First Selection and Random Selection by the same percentage of 50 %. In Long Job First Selection, for each OS entry, the job which has more remaining time to be done will be selected. The remaining time for each job is the sum of its not processed operations' working time. In JSSP where there is no MS, the OS of the first generation is initialized randomly.

In MSOS representation each part has its own genetic operators. For the MS part we incorporate Two-Point Cross-

over used by Watanabe et al. [42] and Uniform Crossover presented by Gao et al. [13] by the same chance of 50 %. Precedence preserving Order-based Crossover (POX) proposed by Lee et al. [28] is also used for OS part. The parents selection is as follows: one parent is randomly selected among the top best individuals, a proportion of the whole population denoted by parameter *TopBest*. The second parent is selected among the whole population randomly.

Mutation operation for MS entries is done by selecting another applicable machine which has the minimum processing time among others. Swapping of two OS entries is the mutation for the OS part. The rates of crossover and mutation are managed using two parameters *CrossoverProbability* and *MutationProbability*, respectively.

4.3 Local search

Our proposed local search heuristic is based on critical operations. Critical operations are ones for which any delay in their processing increases the makespan. The main idea behind the heuristic is to rearrange the critical operations on their corresponding machines. This heuristic first identifies critical operations, then it removes one of the critical operations and reassigns it again to find a better position on its corresponding machine. To save the time complexity of the proposed search method it is applied to a portion of the population, denoted by *TopBest* parameter.

In order to identify the critical operations critical paths are defined. A sequence of critical operations starting from time zero to the completion time is called a critical path. A critical path can be found by tracing the schedule from its completion time to the beginning. To do so, the machines which are working until the last time of the schedule are considered as critical machines. Since the makespan is equal to their completion time any delay in processing their last operation will postpone the whole schedule's completion time. Thus the last operations of the critical machines are critical operations. The next critical operation can be one of the following:

1. The machine-preceding operation: if there is no idle time between the completion of machine-preceding operation and the start of the critical operation.
2. The job-preceding operation: if the finish time of the job-preceding operation equals to the start time of the critical operation.
3. Both case 1 and case 2.

This routine continues until time zero is reached.

To show critical operations finding procedure, a sample classical JSSP with four jobs and four machines is considered. The applicable machine for each operation with its corresponding processing time are presented in Table 2. For instance, the third operation of the first job, namely O_{13} , can

Table 2 A sample classical JSSP

	O_1	O_2	O_3	O_4
J_1	$m_3, 2$	$m_2, 3$	$m_4, 5$	$m_1, 3$
J_2	$m_4, 3$	$m_1, 4$	$m_2, 4$	$m_3, 4$
J_3	$m_2, 3$	$m_4, 4$	$m_3, 6$	$m_1, 6$
J_4	$m_3, 6$	$m_1, 2$	$m_2, 5$	$m_4, 3$

be processed only on machine m_4 in 5 time units. Consider the following operation sequence

$$OS = [2, 3, 2, 1, 1, 4, 4, 3, 4, 2, 3, 1, 3, 1, 2, 4]$$

as a sample OS for the problem given above. Since the problem is a classical JSSP, there is no MS part for chromosome representation. The sequence of operations for this sample OS is

$$O_{21} < O_{31} < O_{22} < O_{11} < O_{12} < O_{41} < O_{42} < O_{32} \\ < O_{43} < O_{23} < O_{33} < O_{13} < O_{34} < O_{14} < O_{24} < O_{44}$$

which is depicted in Fig. 1. Both the first and the third machines are working until the schedule completion time. This means that their last operations, namely O_{14} and O_{24} , are critical operations. Consequently, there are at least two critical paths. The preceding critical operations for O_{14} and O_{24} are a machine-preceding operation and a job-preceding operation, namely O_{34} and O_{23} , respectively. The routine for finding the critical operations continues until time zero is reached. Thus the critical paths are as follows. Based on critical paths there are 9 critical operations in this schedule.

1. $O_{11} \rightarrow O_{41} \rightarrow O_{33} \rightarrow O_{34} \rightarrow O_{14}$
2. $O_{11} \rightarrow O_{41} \rightarrow O_{42} \rightarrow O_{43} \rightarrow O_{23} \rightarrow O_{24}$

In order to improve the search heuristic performance, only the critical operations which satisfy the two following criteria will be selected for reassigning:

1. The critical operation is assigned in the middle of the critical path, not at the first or last.
2. Its preceding and proceeding critical operations on the critical path are either its job-preceding and machine-

proceeding or its machine-preceding and job-proceeding, respectively.

Among those 9 critical operations only 5 operations satisfy these criteria which are O_{41} , O_{33} , O_{34} , O_{43} , and O_{23} . We call these operations selected critical operations (SCOs). The SCOs will be repositioned in the corresponding OS between their job-preceding operation positions and their job-proceeding operation positions. For instance, the SCO O_{33} can be repositioned between the positions of O_{32} and O_{34} , while SCO O_{34} can be reassigned to the positions from O_{33} 's position to the end of OS. Therefore, the possible OSs for the new position of O_{34} are as follows:

$$\{O_{21}, O_{31}, \dots, O_{33}, \mathbf{O}_{34}, O_{13}, O_{14}, O_{24}, O_{44}\} \\ \{O_{21}, O_{31}, \dots, O_{33}, O_{13}, \mathbf{O}_{34}, O_{14}, O_{24}, O_{44}\} \\ \{O_{21}, O_{31}, \dots, O_{33}, O_{13}, O_{14}, \mathbf{O}_{34}, O_{24}, O_{44}\} \\ \{O_{21}, O_{31}, \dots, O_{33}, O_{13}, O_{14}, O_{24}, \mathbf{O}_{34}, O_{44}\} \\ \{O_{21}, O_{31}, \dots, O_{33}, O_{13}, O_{14}, O_{24}, O_{44}, \mathbf{O}_{34}\}$$

Since operation O_{34} and O_{13} are from different jobs and have to be processed on different machines, their orders on the OS does not make any difference on the schedule. In this case, the first two OSs are equivalent, and their schedule is represented in Fig. 1. Again since there is a similar situation for operations O_{24} , O_{34} , and O_{44} , their orders are not important. The last three OSs are equivalent too and their schedule is depicted in Fig. 2.

Among the operations from O_{33} to end of OS, there is no operation of the third job and operation O_{14} is the only operation which has to be processed on the same machine as operation O_{34} . Thus there is only two different schedules: one which considers O_{34} before O_{14} , and another which assigns O_{34} after O_{14} . Since the former is the current schedule, there is only one new OS which assigns O_{34} exactly after O_{14} .

$$\{O_{21}, O_{31}, \dots, O_{33}, O_{13}, \mathbf{O}_{14}, \mathbf{O}_{34}, O_{24}, O_{44}\}$$

Since schedules are independent of the order of operations which are not from the same job and not to be processed on the same machine, the number of new OSs for reassigning a SCO is limited. This number for a SCO equals to the number of operations which have to be processed on the same machine

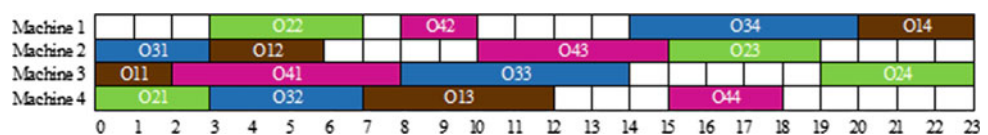
Fig. 1 A sample schedule for sample JSSP to show critical paths**Fig. 2** The sample schedule after repositioning O_{34} 

Table 3 The average numbers of SCOs for Lawrence [27] test problems

Problem	la1	la2	la3	la4	la5	la6	la7	la8	la9	la10
Average	0.26	3.55	3.27	3.28	0.64	0.74	2.84	3.46	2.08	1.42
Problem	la11	la12	la13	la14	la15	la16	la17	la18	la19	la20
Average	1.86	2.71	3.15	0.32	3.31	4.99	6.13	4.87	8.47	7.64
Problem	la21	la22	la23	la24	la25	la26	la27	la28	la29	la30
Average	7.59	4.68	12.38	8.24	8.63	8.20	9.01	9.42	7.08	5.98
Problem	la31	la32	la33	la34	la35	la36	la37	la38	la39	la40
Average	7.26	7.21	9.38	9.10	6.17	9.47	10.03	9.81	7.66	9.68

as the SCO between the positions of its job-preceding operation and its job-proceeding operation on the corresponding OS. So using this routine, we have the following situations:

1. No new OS; that is there is no new position. So the LS cannot alter the schedule for this SCO. Consider operation O_{33} which is a SCO that have to be processed on machine m_3 . In the range between operations O_{32} and O_{34} , there are the following operations:

$$\{O_{43}, O_{23}, O_{33}, O_{13}, O_{14}\}$$

None of these operations have to be processed on machine m_3 . Thus, there is no new position for operation O_{33} .

2. One new OS; it means there is only one new position. So the LS generates a new individual by repositioning the SCO. This is the same routine that has been illustrated for operation O_{34} .
3. Two or more OSs; Since there is more than one OS, an OS is selected randomly and a new individual is generated.

It should be mentioned here that in FJSSs, since the SCOs can be processed on different machines, in order to find the new OSs all the positions for all the applicable machine should be evaluated. Consider the case where operation O_{33} can be processed on machine m_2 . In that range, there are two operations O_{43} and O_{23} that have to be processed on machine m_2 . It means there are two new OSs when operation O_{33} is processed on machine m_2 . In this case, not only the OS is changed, but also the MS is altered.

Overall, our proposed LS works as follows for each individual: It starts with finding SCOs. Then for each SCO, it generates a new individual using the new OS if applicable. So there would be a number of new individuals which is equal to the number of SCOs for each individual. Between these new individuals, the best one which has the best fitness value will be added to the population. Since the LS is only applied to the best individuals of each generation its time complexity depends on both parameters *PopSize* and *TopBest*.

Therefore, the time complexity of the proposed LS is

$$TimeComplexity = IterationNo \times (TopBest \times PopSize) \times Average(SCO\#)$$

where *SCO#* denotes the number of SCOs for each individual.

In order to estimate the time complexity a population of 10,000 random individuals is selected and the average number of SCOs over that population is calculated, which is represented in Table 3. The maximum number of these average values is 12.38 which is the average value for problem la23. Therefore, the proposed local search time complexity for Lawrence [27] test problems would be:

$$TimeComplexity = 12.38 \times IterationNo \times (TopBest \times PopSize) \quad (1)$$

4.4 Priority-based fitness function

In the case where two different individuals have the same fitness value, we do not resort to random selection. Instead, we evaluate different parameters of those individuals and select the one which is more likely to be able to direct the search process toward a better solution. In other words, it would be better to select the individual which is closer to a better solution. For example, consider the two sample schedules before and after repositioning of operation O_{34} . The makespans of both schedules are the same, but the numbers of critical machines are different. The former schedule has two critical machines and the latter has only one. So it is more likely for the latter to become a better solution just by one more repositioning. Therefore, it is better to select the latter schedule than the former if we have to select only one of them. By applying the LS to this schedule, we will have a schedule with a better makespan, represented in Fig. 3.

Based on this procedure a new fitness function is introduced. Since the proposed fitness function calculates different parameters with different priority, it is called Priority-Based Fitness Function (PBFF). The first priority of the PBFF is the main objective of the system. For example, if the optimization goal is to minimizing makespan, the first priority is thus the makespan; the lower, the better. In multi-objective

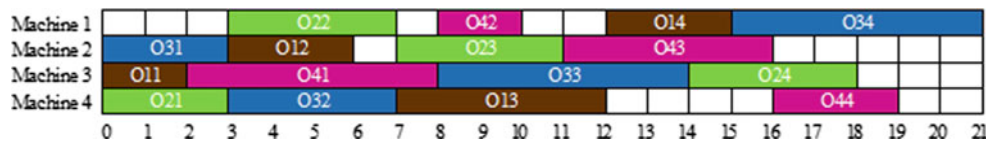


Fig. 3 The sample schedule after repositioning O_{34} , followed by repositioning O_{23}

JSSPs, the first priority of the fitness function is the same as the function of those objectives which is defined by the problem domain. Xing et al. [43], for instance, considered the following function as the multi-objective function with different weights:

$$F(s) = 0.5 \times F_1(s) + 0.2 \times F_2(s) + 0.3 \times F_3(s) \quad (2)$$

where $F(s)$ is the fitness value for schedule s ; $F_1(s)$, $F_2(s)$, and $F_3(s)$ are makespan, total workload and maximum workload of schedule s , respectively.

If we do not consider the next priorities, the PBFF works as the traditional fitness function. The next priorities are therefore considered to direct the search process when a tie occurs. We consider the number of critical machines as the second priority which is very important to the effectiveness of the algorithm. The lower the number of critical machines, the greater probability to reach a new schedule with a better makespan. The third priority which is applicable only for FJSS is the maximum workload. If there are two schedules with the same makespan, the one which has lower maximum workload is more likely to be the direction to the optimal solution.

Therefore, each schedule has three fitness values with different priorities. The comparison between two different schedules is done based on the priorities of each value. First, the values of the highest priority are compared. If they are not equal, the schedule with the lower value is selected as the better schedule. Otherwise, the values for the next priority will be examined and it continues to the last priority. If the values of all priorities are equal, the better schedule is selected randomly.

4.5 MA framework

The framework of our proposed MA presented in Fig. 4 is as follows. The proposed algorithm starts by the initial generation P_0 with a fixed number of individuals. Then, for a fixed number of iterations denoted by *IterationNo*, the genetic operators and local search method are applied to generate new generations.

In each iteration, all the individuals are sorted based on their fitness value calculated by PBFF. Then, genetic operators are applied to generate offspring. The current population and the new one are denoted by P_t and P_{t+1} , respectively. At the next step, the LS is applied to the best individuals of

PROCEDURE: MA Framework

INPUT: Test Problems and Algorithm Parameters

OUTPUT: Optimal or Near-Optimal Schedules

BEGIN

Generate the initial population P_0

FOR (*IterationNo*)

Evaluate current population P_t and sort it

Apply crossover and mutation to generate P_{t+1}

Apply local search method on the *BestTop*

individuals of both P_{t+1} to generate S_{t+1}

Finalized P_{t+1} with the best individuals

END

Output the best individual

END

Fig. 4 MA framework

the new population P_{t+1} to generate a small search population S_{t+1} . The best individuals are a top proportion of a sorted population. Finally, the new population P_{t+1} is finalized with its own best individuals and those resulting from the search heuristic.

It should be mentioned here that in order to show the efficiency of the GA itself the proposed method has been applied to some test problems without using the LS. In those cases, there is no search population.

5 Results

The proposed algorithm is implemented using *Java* version 1.6.0.18 on a system with *Intel(R) Core(TM)2 Quad* 2.50 GHz *CPU* and 8.00 GB *RAM*. Since the proposed method was applied to various test problems with different sizes, we set the population size up to 1000 and run the experiments for up to 200 iterations. The bests individuals are defined as the top 10 % of a sorted population. The crossover and mutation probabilities are set to 90 and 5 %, respectively.

In order to show the efficiency of the proposed algorithm it has been applied to different types of job shop scheduling including JSSP, FJSS and MO-FJSS. Lawrence's benchmark [27] is considered as a dataset for JSSP, which includes 40 case problems ranging from 10 jobs on 5 machines to 30 jobs on 10 machines. The Brandimarte's dataset [5], denoted by *BRData*, has been used as a benchmark for FJSS, which consists of 10 different case problems ranging from 10 jobs on 6 machines to 20 jobs on 15 machines.

To show the effectiveness of the proposed Priority-Based Fitness Function two versions of the proposed method are

Fig. 5 The best (*main bar*) and the average (*extended portion*) values of 10 runs using the traditional fitness function versus PBFF on Lawrence's dataset [27]

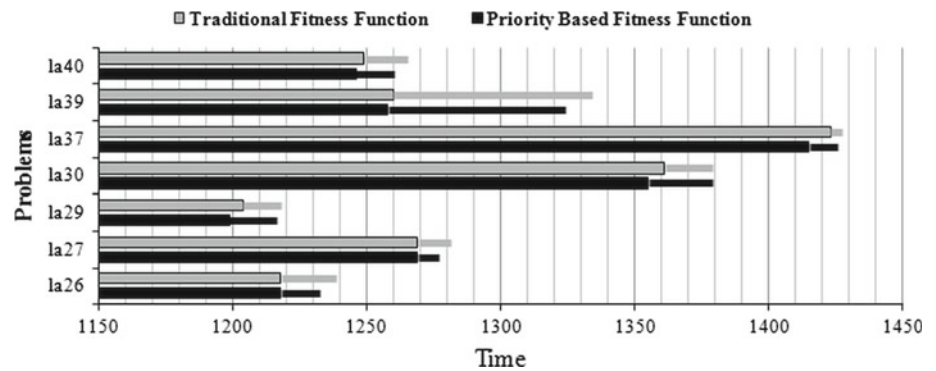
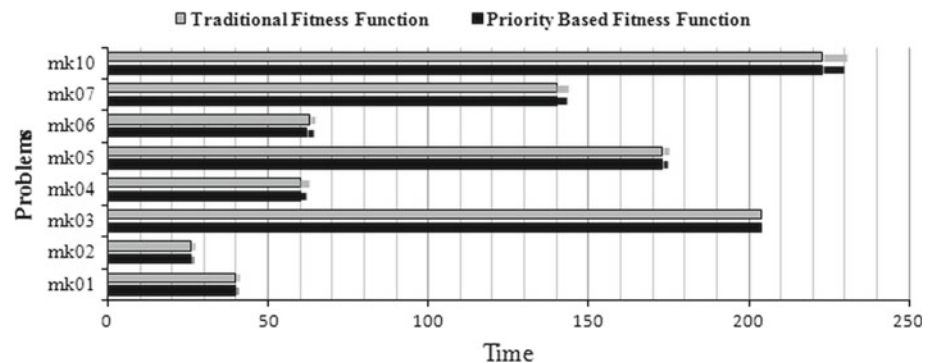


Fig. 6 The best (*main bar*) and the average (*extended portion*) values of 100 runs using the traditional fitness function versus PBFF on BRData [5]



applied to the test problems, one with the traditional fitness function and another with the PBFF. The results of these applications on some JSSPs from Lawrence's dataset [27] have been depicted in Fig. 5. This figure shows the best and average result of 10 different runs using population size of 500 for 100 iterations. The bars show the best solutions and the extensions illustrate the averages. As can be seen clearly in Fig. 5, the PBFF offers better solutions in almost all the sample problems. Moreover, in the cases that both methods find the optimal solutions, the PBFF offers the lower average hence a better convergence rate.

This experiment is also done on the FJSS problems from BRData [5]. Since the results for these test problems are very close to the best solutions, the experiment includes 100 different runs using population size of 1000 in 200 iterations. Figure 6 depicts the best and average results of the experiment. As the figure illustrates, incorporating PBFF offers better solution for problem mk06 and makes better average for others. For problem mk03, since both methods find the optimal solution in all runs, the average is the same as the best result. Considering all the results into account, we can say that the PBFF compared to the traditional fitness function offers better solutions as well as a better convergence rate.

After evaluating the effectiveness of the proposed fitness function, the performance of the proposed LS is examined. So the proposed MA as well as the proposed GA itself without LS are applied to Lawrence's benchmark [27]. The results have been presented in Table 4 which show that LS heuristic improves the overall results. In all the 40 test problems, the

proposed MA has the same or better results than the GA itself. So, it is possible to say that the proposed MA outperforms the proposed GA in all the test problems. The proposed GA can find the optimal solution for 27 problems out of 40, and the proposed MA found the best solution for two more problems. Moreover, the average, standard deviation, and median show that incorporating LS makes the proposed method more stable to find optimal or near-optimal solutions. In other words, it improves the convergence rate of the GA.

To compare the results, the best-known solutions and the results of another MA recently proposed by Hasan et al. [19] have been presented in Table 4 as well. Clearly, our proposed GA without LS outperforms their MA in 39 test problems out of 40. Only on test problem *la39* we observe their result to be better than the result of our proposed GA. However, our proposed MA outperforms their MA for all the 40 test problems.

Furthermore, regarding algorithm efficiency, our proposed method outperforms Hasan et al.'s method [19]. As mentioned before, the time complexity of our proposed local search on Lawrence's benchmark [27] can be calculated using Eq. 1. Based on values 1000 and 200 for *PopSize* and *IterationNo*, the *TimeComplexity* would be less than 250,000 fitness evaluations. Therefore, the overall time complexity of our proposed method would be less than 450,000 fitness evaluations, while the time complexity of their method is 2,500,000 fitness evaluations. In other words, our proposed method offers better solutions while it only needs less than $\frac{1}{5}$ of the fitness evaluations used by their method.

Table 4 Results on Lawrence's benchmark [27] (la01–la40)

Algorithm	Problem	Best	Average	SD	Median	Worst	Problem	Best	Average	SD	Median	Worst
MA [19]	la01	666	667.60	2.90	667	678	la02	655	656.27	3.39	655	666
Proposed GA	10×5	666	666.00	0.00	666	666	10×5	655	658.22	3.66	657	669
Proposed MA	666	666	666.00	0.00	666	666	655	655	655.00	0.00	655	655
MA [19]	la03	597	613.93	7.63	617	619	la04	590	593.33	2.44	595	595
Proposed GA	10×5	597	606.40	9.34	603	617	10×5	590	592.92	2.60	593	602
Proposed MA	597	597	597.00	0.00	597	597	590	590	590.60	1.26	590	593
MA [19]	la05	593	593.00	0.00	593	593	la06	926	926.00	0.00	926	926
Proposed GA	10×5	593	593.00	0.00	593	593	15×5	926	926.00	0.00	926	926
Proposed MA	593	593	593.00	0.00	593	593	926	926	926.00	0.00	926	926
MA [19]	la07	890	890.00	0.00	890	890	la08	863	863.00	0.00	863	863
Proposed GA	15×5	890	890.00	0.00	890	890	15×5	863	863.00	0.00	863	863
Proposed MA	890	890	890.00	0.00	890	890	863	863	863.00	0.00	863	863
MA [19]	la09	951	951.00	0.00	951	951	la10	958	958.00	0.00	958	958
Proposed GA	15×5	951	951.00	0.00	951	951	15×5	958	958.00	0.00	958	958
Proposed MA	951	951	951.00	0.00	951	951	958	958	958.00	0.00	958	958
MA [19]	la11	1222	1222.00	0.00	1222	1222	la12	1039	1039.00	0.00	1039	1039
Proposed GA	20×5	1222	1222.00	0.00	1222	1222	20×5	1039	1039.00	0.00	1039	1039
Proposed MA	1222	1222	1222.00	0.00	1222	1222	1039	1039	1039.00	0.00	1039	1039
MA [19]	la13	1150	1150.00	0.00	1150	1150	la14	1292	1292.00	0.00	1292	1292
Proposed GA	20×5	1150	1150.00	0.00	1150	1150	20×5	1292	1292.00	0.00	1292	1292
Proposed MA	1150	1150	1150.00	0.00	1150	1150	1292	1292	1292.00	0.00	1292	1292
MA [19]	la15	1207	1207.13	0.52	1207	1209	la16	945	968.27	15.46	979	982
Proposed GA	20×5	1207	1207.00	0.00	1207	1207	10×10	945	964.85	15.33	966	982
Proposed MA	1207	1207	1207.00	0.00	1207	1207	945	945	952.40	8.72	950	973
MA [19]	la17	784	788.93	4.18	792	793	la18	848	859.27	4.57	861	861
Proposed GA	10×10	784	784.81	2.44	784	804	10×10	848	859.37	4.55	861	876
Proposed MA	784	784	784.00	0.00	784	784	848	848	848.50	1.41	848	852
MA [19]	la19	842	855.47	7.76	855	869	la20	907	910.00	2.54	912	912
Proposed GA	10×10	842	859.34	8.09	860	876	10×10	907	910.76	3.46	911	932
Proposed MA	842	842	849.00	3.13	849	854	902	902	906.90	2.13	907	911
MA [19]	la21	1079	1097.60	12.48	1096	1124	la22	960	981.00	13.58	979	1000
Proposed GA	15×10	1055	1074.22	11.69	1075	1116	15×10	937	954.28	8.53	951	981
Proposed MA	1046	1053	1058.20	4.02	1059	1067	927	927	934.20	4.54	933.5	941
MA [19]	la23	1032	1032.00	0.00	1032	1032	la24	959	996.40	16.21	1003	1011
Proposed GA	15×10	1032	1032.20	0.99	1032	1038	15×10	950	970.41	10.28	967	998
Proposed MA	1032	1032	1032.00	0.00	1032	1032	935	941	954.70	11.61	953	967
MA [19]	la25	991	1016.67	19.64	1013	1076	la26	1218	1234.27	16.12	1228	1266
Proposed GA	15×10	990	1012.13	12.40	1009	1039	20×10	1218	1246.57	13.98	1245	1285
Proposed MA	977	984	986.90	2.60	986	992	1218	1218	1218.00	0.00	1218	1218
MA [19]	la27	1286	1306.33	13.17	1301	1333	la28	1286	1306.33	13.17	1301	1333
Proposed GA	20×10	1269	1295.13	10.76	1294	1326	20×10	1229	1262.12	11.75	1262	1296
Proposed MA	1235	1256	1267.10	6.44	1269	1275	1216	1223	1231.80	6.01	1234	1241
MA [19]	la29	1221	1240.47	6.98	1240.5	1249	la30	1355	1362.33	8.23	1359.5	1380
Proposed GA	20×10	1201	1229.14	10.61	1228	1250	20×10	1355	1376.06	13.59	1374	1409
Proposed MA	1157	1187	1196.40	6.10	1198	1206	1355	1355	1355.00	0.00	1355	1355
MA [19]	la31	1784	1784.00	0.00	1784	1784	la32	1850	1850.00	0.00	1850	1850
Proposed GA	30×10	1784	1784.00	0.00	1784	1784	30×10	1850	1850.41	1.19	1850	1856
Proposed MA	1784	1784	1784.00	0.00	1784	1784	1850	1850	1850.00	0.00	1850	1850

Table 4 continued

Algorithm	Problem	Best	Average	SD	Median	Worst	Problem	Best	Average	SD	Median	Worst
MA [19]	la33	1719	1719.00	0.00	1719	1719	la34	1721	1721.00	0.00	1721	1721
Proposed GA	30×10	1719	1719.00	0.00	1719	1719	30×10	1721	1734.29	6.84	1734	1750
Proposed MA	1719	1719	1719.00	0.00	1719	1719	1721	1721	1721.00	0.00	1721	1721
MA [19]	la35	1888	1888.00	0.00	1888	1888	la36	1307	1328.67	11.59	1327	1346
Proposed GA	30×10	1888	1888.02	0.14	1888	1889	15×15	1291	1310.97	10.06	1308.5	1334
Proposed MA	1888	1888	1888.00	0.00	1888	1888	1268	1276	1285.70	10.50	1278	1300
MA [19]	la37	1442	1473.60	11.51	1479	1487	la38	1266	1309.13	14.52	1314	1329
Proposed GA	15×15	1425	1434.90	18.81	1425	1483	15×15	1219	1252.20	14.36	1252	1304
Proposed MA	1397	1401	1420.40	8.68	1425	1425	1196	1208	1218.30	8.07	1218	1238
MA [19]	la39	1252	1282.60	16.62	1277	1301	la40	1252	1279.60	17.84	1289	1303
Proposed GA	15×15	1258	1276.76	9.79	1275	1309	15×15	1244	1266.91	7.66	1269	1287
Proposed MA	1233	1240	1251.30	4.81	1251	1258	1222	1233	1246.70	7.18	1246.5	1259

The optimal solutions are illustrated in bold

The proposed method is also compared with other algorithms for classical JSSP including our previous work [33], the heuristic method introduced by Adams et al. [1] and the hybrid GA proposed by Gonçalves et al. [15]. This comparison has been represented in Table 5. The optimal solutions are illustrated in bold face, and asterisk (*) determines the best solutions found by compared algorithms. All the results of our proposed method received a bold face or asterisk except two problems: la21 and la25. This means that the proposed method outperforms all the mentioned algorithms by offering the best solution in almost all test problems. To be able to compare the algorithms over all 40 problems, an error rate (ER) is defined as follows.

$$ER = \frac{C - LB}{LB} \times 100 \%$$

where LB is the best-known solution, and C is the best solution found by the algorithms. Table 5 represents the ER values in brackets. As illustrated in this table, our proposed method has the minimum average of ER values for all the 40 test problems. To show more comprehensive comparisons, the ER values corresponding to the most challenging test problems have been illustrated in Fig. 7.

Furthermore, our proposed MA is also evaluated for FJSS. The results have been presented in Table 6. The results are compared to the state-of-the-art approaches including Hybrid GA proposed by Gao et al. [13], AIA proposed by Bagheri et al. [3], three Genetic Algorithms proposed by Chen et al. [8], Pezzella et al. [40] and Zhang et al. [45], and Clonal Selection approach suggested by Ong et al. [37]. The results show that our proposed method offers competitive solutions compared to the existing state-of-the-art methods in FJSS. The proposed method finds the best solution for 8 problems out of the 10. Though Ong et al. [37] claimed that they found 39 as the makespan for test problem *mk01*, the makespan of

39 for this test problem is not feasible and the best solution is 40. This issue has been proved in Appendix A.

The proposed MA was also applied in MO-FJSS problems. The benchmarks presented in Xing et al. [43] are considered as test problems. The scheduling goal is to minimize a multi-objective function with different weight represented in Eq. 2. The results of both our proposed MA and their algorithm have been presented in Table 7. Our proposed MA found the best solutions for all the problems. Moreover, it improves the solution for the 4th benchmark which is represented in bold face in Table 7. The solution found for this benchmark, whose problem specification is presented in Table 1, has been depicted in Fig. 8.

6 Conclusions

In this article, a memetic algorithm is proposed to deal with different types of JSSPs. The main contributions of our proposed methods are the introduction of the priority-based fitness function and a new heuristic for local search approaches.

The experiment results show that incorporating our proposed fitness function improves the final solutions by saving the individuals which have better characteristics. The proposed GA method itself without the LS outperforms the MA recently proposed by Hasan et al. [19] for 39 test problems out of 40 in JSSP. Moreover, the proposed GA is improved by incorporating the LS, such that it finds the optimal solutions for two more test problems. The better statistical information for MA such as standard deviation determines that the LS helps the GA to offers more stable results, which means it helps the GA to converge faster.

The proposed method is also effective in other types of JSSP. Compared to the state-of-the-art methods in FJSS, our proposed method offers competitive results by obtaining the optimal solutions for 8 test problems and finding the

Table 5 Comparison of different types of algorithms on Lawrence's benchmark [27]

Problem	Best known	Heuristic [1]	Hybrid GA [15]	Hassan et al. MA [19]	Our previous MA [33]	Proposed MA
LA01	666	666 (0.00 %)	666 (0.00 %)	666 (0.00 %)	666 (0.00 %)	666 (0.00 %)
LA02	655	669 (2.14 %)	655 (0.00 %)	655 (0.00 %)	655 (0.00 %)	655 (0.00 %)
LA03	597	605 (1.34 %)	597 (0.00 %)	597 (0.00 %)	597 (0.00 %)	597 (0.00 %)
LA04	590	593 (0.51 %)	590 (0.00 %)	590 (0.00 %)	590 (0.00 %)	590 (0.00 %)
LA05	593	593 (0.00 %)	593 (0.00 %)	593 (0.00 %)	593 (0.00 %)	593 (0.00 %)
LA06	926	926 (0.00 %)	926 (0.00 %)	926 (0.00 %)	926 (0.00 %)	926 (0.00 %)
LA07	890	890 (0.00 %)	890 (0.00 %)	890 (0.00 %)	890 (0.00 %)	890 (0.00 %)
LA08	863	863 (0.00 %)	863 (0.00 %)	863 (0.00 %)	863 (0.00 %)	863 (0.00 %)
LA09	951	951 (0.00 %)	951 (0.00 %)	951 (0.00 %)	951 (0.00 %)	951 (0.00 %)
LA10	958	959 (0.10 %)	958 (0.00 %)	958 (0.00 %)	958 (0.00 %)	958 (0.00 %)
LA11	1222	1222 (0.00 %)	1222 (0.00 %)	1222 (0.00 %)	1222 (0.00 %)	1222 (0.00 %)
LA12	1039	1039 (0.00 %)	1039 (0.00 %)	1039 (0.00 %)	1039 (0.00 %)	1039 (0.00 %)
LA13	1150	1150 (0.00 %)	1150 (0.00 %)	1150 (0.00 %)	1150 (0.00 %)	1150 (0.00 %)
LA14	1292	1292 (0.00 %)	1292 (0.00 %)	1292 (0.00 %)	1292 (0.00 %)	1292 (0.00 %)
LA15	1207	1207 (0.00 %)	1207 (0.00 %)	1207 (0.00 %)	1207 (0.00 %)	1207 (0.00 %)
LA16	945	978 (3.49 %)	945 (0.00 %)	945 (0.00 %)	945 (0.00 %)	945 (0.00 %)
LA17	784	787 (0.38 %)	784 (0.00 %)	784 (0.00 %)	784 (0.00 %)	784 (0.00 %)
LA18	848	859 (1.30 %)	848 (0.00 %)	848 (0.00 %)	848 (0.00 %)	848 (0.00 %)
LA19	842	860 (2.14 %)	842 (0.00 %)	842 (0.00 %)	842 (0.00 %)	842 (0.00 %)
LA20	902	914 (1.33 %)	907 (0.55 %)	907 (0.55 %)	907 (0.55 %)	902 (0.00 %)
LA21	1046	1084 (3.63 %)	1046 (0.00 %)	1079 (3.15 %)	1057 (1.05 %)	1053 (0.67 %)
LA22	927	944 (1.83 %)	935 (0.86 %)	960 (3.56 %)	935 (0.86 %)	927 (0.00 %)
LA23	1032	1032 (0.00 %)	1032 (0.00 %)	1032 (0.00 %)	1032 (0.00 %)	1032 (0.00 %)
LA24	935	976 (4.39 %)	953 (1.93 %)	959 (2.57 %)	944 (0.96 %)	941 (0.64 %)*
LA25	977	1017 (4.09 %)	986 (0.92 %)	991 (1.43 %)	983 (0.61 %)*	984 (0.72 %)
LA26	1218	1224 (0.49 %)	1218 (0.00 %)	1218 (0.00 %)	1218 (0.00 %)	1218 (0.00 %)
LA27	1235	1291 (4.53 %)	1256 (1.70 %)*	1286 (4.13 %)	1269 (2.75 %)	1256 (1.70 %)*
LA28	1216	1250 (2.80 %)	1232 (1.32 %)	1286 (5.76 %)	1223 (0.58 %)*	1223 (0.58 %)*
LA29	1157	1239 (7.09 %)	1196 (3.37 %)	1221 (5.53 %)	1191 (2.94 %)	1187 (2.59 %)*
LA30	1355	1355 (0.00 %)	1355 (0.00 %)	1355 (0.00 %)	1355 (0.00 %)	1355 (0.00 %)
LA31	1784	1784 (0.00 %)	1784 (0.00 %)	1784 (0.00 %)	1784 (0.00 %)	1784 (0.00 %)
LA32	1850	1850 (0.00 %)	1850 (0.00 %)	1850 (0.00 %)	1850 (0.00 %)	1850 (0.00 %)
LA33	1719	1719 (0.00 %)	1719 (0.00 %)	1719 (0.00 %)	1719 (0.00 %)	1719 (0.00 %)
LA34	1721	1721 (0.00 %)	1721 (0.00 %)	1721 (0.00 %)	1721 (0.00 %)	1721 (0.00 %)
LA35	1888	1888 (0.00 %)	1888 (0.00 %)	1888 (0.00 %)	1888 (0.00 %)	1888 (0.00 %)
LA36	1268	1305 (2.92 %)	1279 (0.87 %)	1307 (3.08 %)	1281 (1.03 %)	1276 (0.63 %)*
LA37	1397	1423 (1.86 %)	1408 (0.79 %)	1442 (3.22 %)	1429 (2.29 %)	1401 (0.29 %)*
LA38	1196	1255 (4.93 %)	1219 (1.92 %)	1266 (5.85 %)	1208 (1.00 %)*	1208 (1.00 %)*
LA39	1233	1273 (3.24 %)	1246 (1.05 %)	1252 (1.54 %)	1248 (1.22 %)	1240 (0.57 %)*
LA40	1222	1269 (3.85 %)	1241 (1.55 %)	1252 (2.45 %)	1234 (0.98 %)	1233 (0.90 %)*
Average error rate		1.46 %	0.42 %	1.07 %	0.42 %	0.26 %

near-optimal solutions for the rest two problems. In addition, the application of the proposed method in MO-FJSS shows that it is efficient for these problems as well. Our proposed method finds the best solutions for all the test problems and

more importantly it improves the final solution for one test problem.

The application of the proposed algorithm on Stochastic Flexible Job Shop Scheduling will be considered in future

Fig. 7 Comparison of different types of algorithms on Lawrence's benchmark [27]

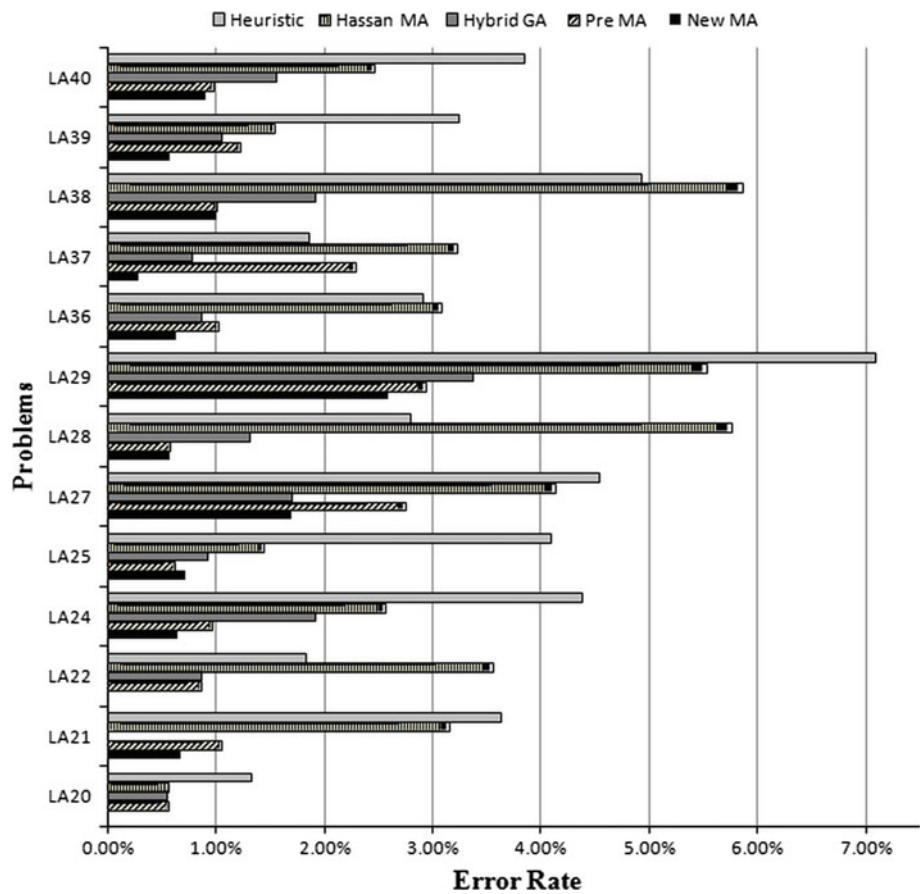


Table 6 Comparison of different types of algorithms on BRData [5]

Problem	Size	hGA [13]	AIA [3]	GA Chen [8]	GA Pezzella [40]	GA Zhang [45]	Clonal [37]	Proposed
mk01	10 × 6	40	40	40	40	40	39	40
mk02	10 × 6	26	26	29	26	26	27	26
mk03	15 × 8	204	204	204	204	204	—	204
mk04	15 × 8	60	60	63	60	60	65	60
mk05	15 × 4	172	173	181	173	173	173	172
mk06	10 × 15	58	63	60	63	58	70	59
mk07	20 × 5	139	140	148	139	144	145	139
mk08	20 × 10	523	523	523	523	523	523	523
mk09	20 × 10	307	312	308	311	307	311	307
mk10	20 × 15	197	214	212	212	198	—	216

Bold values represent the optimal solutions for the corresponding test problems

Table 7 Results on FJSS benchmarks used by Xing et al. [43]

Benchmarks	Xing et al. [43]				Proposed MA			
	F_1	F_2	F_3	F	F_1	F_2	F_3	F
Instance 1	12	32	8	14.8	12	32	8	14.8
Instance 2	14	77	12	26	14	77	12	26
Instance 3	11	62	10	20.9	11	62	10	20.9
Instance 4	7	42	6	13.7	7	43	5	13.6
Instance 5	11	91	11	27	11	91	11	27

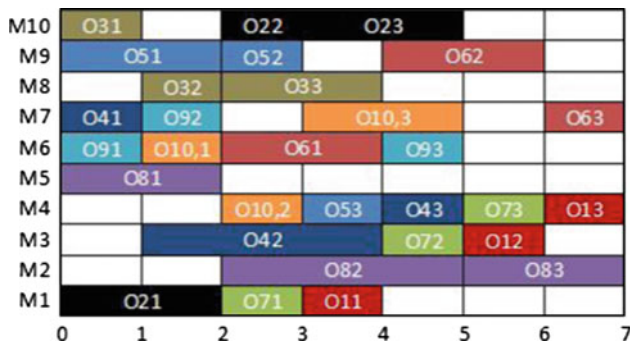


Fig. 8 Results on 4th FJSS benchmark used by Xing et al. [43]

work. Moreover, the parameters of the algorithm have not been investigated to find the best set of parameters.

Acknowledgments This work was supported in part by a grant from the NSERC Discovery No. 327482.

Appendix A

Test problem *mk01* is one of the *BRData* problems published by Brandimarte [5]. The best makespan found for this test problem by the proposed methods and almost all the existing methods is 40, while Ong et al. [37] claimed that they found 39. This Appendix is to prove that the makespan of 39 for this test problem is not feasible.

Claim *The makespan of 39 or less for test problem mk01 of BRData is not feasible.*

Proof The specification of problem *mk01* which is a sample of P-FJSS problems has been presented in Table 8. It consists of 10 different jobs including either 5 or 6 operations which should be processed on 6 different machines. For each operations, the machines which are not applicable are denoted by X.

As can be seen in Table 8, there is no flexibility for operations O_{21} , O_{31} , O_{42} , O_{53} , O_{64} and O_{84} . They are all supposed to be processed on machine m_2 . The processing times for all of them are the same as 6 time units. So, machine m_2 is supposed to work on these operations from time 0 to 36. It means that one of these operations has to be processed in time interval (30–36). As all of these operations are not the last operations of their job, they have one or more job-proceeding operations. The minimum processing time needed for the job-proceeding operations of operations O_{21} , O_{31} , O_{42} , O_{53} , O_{64} and O_{84} are 10, 8, 4, 14, 3 and 6, respectively. So, if there is any schedule with makespan 39, it should assign operation O_{64} in the time interval (30–36), and assign O_{65} to machine m_1 from time 36 to 37, and O_{66} to machine m_4 from time 37 to 39. Therefore, if it can be proved that machine m_4 is not idle in time interval (37–39), it has been proved that there is

Table 8 Test problem *mk01* of *BRData* [5]

Jobs	Operations	Machines					
		m_1	m_2	m_3	m_4	m_5	m_6
J_1	O_{11}	5	X	4	X	X	X
	O_{12}	X	1	5	X	3	X
	O_{13}	X	X	4	X	X	2
	O_{14}	1	6	X	X	X	5
	O_{15}	X	X	1	X	X	X
	O_{16}	X	X	6	3	X	6
J_2	O_{21}	X	6	X	X	X	X
	O_{22}	X	X	1	X	X	X
	O_{23}	2	X	X	X	X	X
	O_{24}	X	6	X	6	X	X
	O_{25}	1	6	X	X	X	5
	O_{26}	X	X	4	X	X	X
J_3	O_{31}	X	6	X	X	X	X
	O_{32}	X	X	4	X	X	2
	O_{33}	1	6	X	X	X	5
	O_{34}	X	6	4	X	X	6
	O_{35}	1	X	X	X	5	X
	O_{36}	X	X	4	X	X	X
J_4	O_{41}	1	6	X	X	X	5
	O_{42}	X	6	X	X	X	X
	O_{43}	X	X	1	X	X	X
	O_{44}	X	1	5	X	3	X
	O_{45}	X	X	4	X	X	2
	O_{46}	X	X	4	X	X	X
J_5	O_{51}	X	1	5	X	3	X
	O_{52}	1	6	X	X	X	5
	O_{53}	X	6	X	X	X	X
	O_{54}	5	X	4	X	X	X
	O_{55}	X	6	X	6	X	X
	O_{56}	X	6	4	X	X	6
J_6	O_{61}	X	X	4	X	X	2
	O_{62}	2	X	X	X	X	X
	O_{63}	X	6	4	X	X	6
	O_{64}	X	6	X	X	X	X
	O_{65}	1	6	X	X	X	5
	O_{66}	3	X	X	2	X	X
J_7	O_{71}	X	X	X	X	X	1
	O_{72}	3	X	X	2	X	X
	O_{73}	X	6	4	X	X	6
	O_{74}	6	6	X	X	1	X
	O_{75}	X	X	1	X	X	X
	O_{76}	X	X	4	X	X	2
J_8	O_{81}	X	X	4	X	X	2
	O_{82}	X	6	4	X	X	6
	O_{83}	1	6	X	X	X	5
	O_{84}	X	6	X	X	X	X
	O_{85}	X	6	X	6	X	X
	O_{86}	X	X	X	X	X	1
J_9	O_{91}	X	X	X	X	X	1
	O_{92}	1	X	X	X	5	X
J_{10}	O_{101}	X	X	6	3	X	6
	O_{102}	X	X	6	3	X	6

Table 8 continued

Jobs	Operations	Machines					
		m_1	m_2	m_3	m_4	m_5	m_6
J_{10}	O_{94}	2	X	X	X	X	X
	O_{95}	X	6	4	X	X	6
	O_{96}	X	6	X	6	X	X
	O_{101}	X	X	4	X	X	2
	O_{102}	X	6	4	X	X	6
	O_{103}	X	1	5	X	3	X
	O_{104}	X	X	X	X	X	1
J_9	O_{105}	X	6	X	6	X	X
	O_{106}	3	X	X	2	X	X
	O_{91}	X	X	X	X	X	1
	O_{92}	1	X	X	X	5	X
	O_{93}	X	X	6	3	X	6
	O_{94}	2	X	X	X	X	X
	O_{95}	X	6	4	X	X	6
J_{10}	O_{96}	X	6	X	6	X	X
	O_{101}	X	X	4	X	X	2
	O_{102}	X	6	4	X	X	6
	O_{103}	X	1	5	X	3	X
	O_{104}	X	X	X	X	X	1
	O_{105}	X	6	X	6	X	X
	O_{106}	3	X	X	2	X	X

no feasible schedule for problem $mk01$ with makespan 39 or less.

Operations O_{24} , O_{55} , O_{85} , O_{96} and O_{105} have the flexibility to be processed on either machine m_2 or m_4 . Since their processing time on both machines are 6, and machine m_2 is already assigned from time 0 to 36, all the 5 operations should be processed on machine m_4 . Since all the 5 operations are not the first operations of their corresponding jobs, they have a number of job-preceding operations. So, the minimum time needed for the job-preceding operations to be processed for operations O_{24} , O_{55} , O_{85} , O_{96} and O_{105} are 9, 12, 13, 11 and 8, respectively. So, in the best case, machine m_4 should process these operations from time 8 to 38. It means that machine m_4 is not idle from time 37 to 38. Therefore, it is proved that machine m_4 is not idle in the time interval (37–39) and the best starting time for operation O_{66} on machine m_4 is time 38, for which the processing lasts to time 40. Also, O_{66} can be processed on machine m_2 from time 37 to 40. For both possible cases, the makespan is 40, and not less. So, the claim has been proved.

References

- Adams J, Balas E, Zawack D (1988) The shifting bottleneck procedure for job shop scheduling. *Manag Sci* 34(3):391–401
- Asano M, Ohta H (2002) A heuristic for job shop scheduling to minimize total weighted tardiness. *Comput Ind Eng* 42:137–147
- Bagheri A, Zandieh M, Mahdavi I, Yazdani M (2010) An artificial immune algorithm for the flexible job-shop scheduling problem. *Future Gen Comput Syst* 26:533–541
- Becerra RL, Coello CA (2005) A cultural algorithm for solving the job-shop scheduling problem. In: *Knowledge incorporation in evolutionary computation, studies in fuzziness and soft computing*, vol 167. Springer, Berlin, pp 37–55
- Brandimarte P (1993) Routing and scheduling in a flexible job shop by taboo search. *Ann Oper Res* 41:157–183
- Brucker P, Schlie R (1990) Job-shop scheduling with multi-purpose machines. *Computing* 45:369–375
- Caumond A, Lacomme P, Tcherneva N (2008) A memetic algorithm for the job-shop with time-lags. *Comput Oper Res* 35(7):2331–2356
- Chen H, Ihlow J, Lehmann C (1999) A genetic algorithm for flexible job-shop scheduling. In: *IEEE international conference on robotics and automation*, Detroit, pp 1120–1125
- Chiang T-C, Cheng H-C, Fu L-C (2011) NNMA: an effective memetic algorithm for solving multiobjective permutation flow shop scheduling problems. *Exp Syst Appl* 38(5):5986–5999
- Croce FD, Tadei R, Volta G (1995) A genetic algorithm for the job shop problem. *Comput Oper Res* 22:15–24
- Fonseca DJ, Navaresse D (2002) Artificial neural networks for job shop simulation. *Adv Eng Inform* 16(4):241–246
- Gao J, Gen M, Sun L, Zhao X (2007) A hybrid of genetic algorithm and bottleneck shifting for multiobjective flexible job shop scheduling problems. *Comput Ind Eng* 53(1):149–162
- Gao J, Sun L, Gen M (2008) A hybrid genetic and variable neighborhood descent algorithm for flexible job shop scheduling problems. *Comput Oper Res* 35(9):2892–2907
- Garey MR, Johnson DS, Sethi R (1976) The complexity of flow-shop and jobshop scheduling. *Math Oper Res* 1:117–129
- Goncalves JF, de Magalhaes Mendes JJ, Resende MGC (2002) A hybrid genetic algorithm for the job shop scheduling problem. Technical report TD-5EAL6J, AT&T Labs
- Hasan SMK, Sarker R, Cornforth D (2007) Hybrid genetic algorithm for solving job-shop scheduling problem. In: *6th IEEE/ACIS international conference on computer and information science*, Melbourne, pp 519–524
- Hasan SMK, Sarker R, Cornforth D (2007) Modified genetic algorithm for job-shop scheduling: a gap-utilization technique. In: *IEEE congress on evolutionary computation*, Singapore, pp 3804–3811
- Hasan SMK, Sarker R, Cornforth D (2008) GA with priority rules for solving job-shop scheduling problems. In: *IEEE world congress on computational intelligence*, Hong Kong, pp 1913–1920
- Hasan SMK, Sarker R, Essam D, Cornforth D (2008) Memetic algorithms for solving job-shop scheduling problems. *Memetic Comp* 1:69–83
- Huang KL, Liao CJ (2008) Ant colony optimization combined with taboo search for the job shop scheduling problem. *Comput Oper Res* 35(4):1030–1046
- Huang WQ, Yin AH (2004) An improved shifting bottleneck procedure for the job shop scheduling problem. *Comput Oper Res* 31:2093–2110
- Hurink E, Jurisch B, Thole M (1994) Tabu search for the job shop scheduling problem with multi-purpose machines. *Oper Res Spectrum* 15:205–215
- Kacem I, Hammadi S, Borne P (2002) Approach by localization and multi objective evolutionary optimization for flexible job-shop scheduling problems. *IEEE Trans Syst Man Cybernet Part C* 32(1):1–13
- Kacem I, Hammadi S, Borne P (2002) Pareto-optimality approach for flexible job-shop scheduling problems: hybridization of evolu-

- tionary algorithms and fuzzy logic. *J Math Comput Simul* 60:245–276
25. Kolonko M (1999) Some new results on simulated annealing applied to the job-shop scheduling problem. *Eur J Oper Res* 113:123–136
26. Lawrence D (1985) Job shop scheduling with genetic algorithms. In: *First international conference on genetic algorithms*, Mahwah, New Jersey, pp 136–140
27. Lawrence S (1984) Resource constrained project scheduling: an experimental investigation of heuristic scheduling techniques. Master's thesis, Graduate School of Industrial Administration, Carnegie-Mellon University, Pittsburgh, Pennsylvania
28. Lee KM, Yamakawa T, Lee KM (1998) A genetic algorithm for general machine scheduling problems. *Int J Knowl-Based Electron* 2:60–66
29. Mahdavi I, Shirazi B, Solimanpur M (2010) Development of a simulation-based decision support system for controlling stochastic flexible job shop manufacturing systems. *Simul Model Pract Theory* 18:768–786
30. Mattfeld DC, Bierwirth C (2004) An efficient genetic algorithm for job shop scheduling with tardiness objectives. *Eur J Oper Res* 155:616–630
31. Mobini M, Mobini Z, Rabbani M (2011) An artificial immune algorithm for the project scheduling problem under resource constraints. *Appl Soft Comput* 11(2):1975–1982
32. Moscato P (1999) Memetic algorithms: a short introduction. *New Ideas Optim* 14:219–234
33. Raeesi N. MR, Kobti Z (2011) A machine operation lists based memetic algorithm for job shop scheduling. In: *IEEE congress on evolutionary computation (CEC)*, New Orleans, LA, USA, June 2011, pp 2436–2443
34. Naderi B, Fatemi Ghomi SMT, Aminnayeri M, Zandieh M (2011) Scheduling open shops with parallel machines to minimize total completion time. *J Comput Appl Math* 235(5):1275–1287
35. Ombuki BM, Ventresca M (2004) Local search genetic algorithms for the job shop scheduling problem. *Appl Intell* 21:99–109
36. Ong YS, Keane AJ (2004) Meta-lamarckian learning in memetic algorithms. *IEEE Trans Evol Comput* 8:99–110
37. Ong ZX, Tay JC, Kwah CK (2005) Applying the clonal selection principle to find flexible job-shop schedules. In: *LNCS*, vol 3627, pp 442–455
38. Park BJ, Choi HR, Kim HS (2003) A hybrid genetic algorithm for the job shop scheduling problems. *Comput Ind Eng* 45:597–613
39. Pezzella F, Merelli E (2000) A tabu search method guided by shifting bottleneck for the job shop scheduling problem. *Eur J Oper Res* 120:297–310
40. Pezzella F, Morganti G, Ciaschetti G (2008) A genetic algorithm for the flexible jobshop scheduling problem. *Comput Oper Res* 35(10):3202–3212
41. Wang CQ, Cao YF, Dai GZ (2005) Bi-directional convergence aco for job-shop scheduling. *Comput Integr Manuf Syst* 10(7):820–824
42. Watanabe M, Ida K, Gen M (2005) A genetic algorithm with modified crossover operator and search area adaptation for the job-shop scheduling problem. *Comput Ind Eng* 48:743–752
43. Xing L-N, Chen Y-W, Yang K-W (2009) Multi-objective flexible job shop schedule: design and evaluation by simulation modeling. *Appl Soft Comput* 9:362–376
44. Yang J-H, Sun L, Lee HP, Qian Y, Liang Y-C (2008) Clonal selection based memetic algorithm for job shop scheduling problems. *J Bionic Eng* 5(2):111–119
45. Zhang G, Gao L, Shi Y (2011) An effective genetic algorithm for the flexible job-shop scheduling problem. *Exp Syst Appl* 38(4):3563–3573